

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 12
SOLID PRINCIPLES



Disusun Oleh:
ELISABETH NAINGGOLAN
105224037

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. PENDAHULUAN

Praktikum Pemrograman Berorientasi Objek (PBO) pada modul ini berfokus pada pemahaman dan implementasi lima prinsip desain SOLID (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, dan Dependency Inversion) di dalam pemrograman Java. Prinsip SOLID merupakan fondasi dalam rekayasa perangkat lunak guna mentransformasikan kode yang kaku, rapuh, dan sulit diubah menjadi arsitektur aplikasi yang modular, tangkas, serta adaptif terhadap perluasan fitur. Penerapan SOLID memisahkan setiap domain tanggung jawab secara terisolasi melalui pemanfaatan abstraksi antarmuka.

Implementasi studi kasus pada laporan praktikum ini diterapkan pada perancangan ulang sistem portal akademik SIAKAD kampus yang mengalami kegagalan sistem akibat penumpukan beban logika operasional. Sebelum rekonstruksi dilakukan, sistem bertumpu sepenuhnya pada kelas SistemKRSManager yang mengurus validasi prasyarat mata kuliah, kalkulasi Uang Kuliah Tunggal (UKT), pembuatan draf KRS dalam format PDF, hingga manajemen koneksi database secara langsung. Melalui integrasi berlapis dari kelima prinsip SOLID, sistem SIAKAD direkayasa ulang guna menjamin stabilitas aplikasi saat menghadapi beban akses tinggi serta memisahkan ketergantungan keras logika bisnis dari infrastruktur teknis database.

Prinsip Single Responsibility Principle (SRP) diterapkan untuk memecah tanggung jawab SistemKRSManager ke dalam komponen validator akademik dan pencetak draf yang independen. Evaluasi skema perhitungan UKT yang dipenuhi ratusan baris pengkondisian if-else direkonstruksi menggunakan Open/Closed Principle (OCP) berbasis Strategy Pattern agar siap menerima perluasan skema program "Merdeka Belajar Kampus Merdeka" (MBKM) tanpa memodifikasi rumus lama. Penanganan jenis mata kuliah (Teori dan Praktikum) diselesaikan dengan Liskov Substitution Principle (LSP) dan Interface Segregation Principle (ISP) untuk mengeliminasi polusi metode dan mencegah runtime error berupa UnsupportedOperationException. Terakhir, arsitektur MySQL diputus menggunakan Dependency Inversion Principle (DIP) melalui pembuatan abstraksi interface koneksi, sehingga mempermudah proses migrasi ke infrastruktur Cloud NoSQL.

II. VARIABEL

No	Nama Variabel	Tipe Data	Fungsi
1.	db	DatabaseConnection	Menyimpan referensi objek database berbasis abstraksi interface (DIP), menggantikan referensi konkret MySQLDatabaseConnection terdahulu agar logika bisnis tidak terikat pada vendor database tertentu.
	nama	String	Menyimpan informasi nama mata kuliah teori yang digunakan

			saat proses registrasi kelas berlangsung.
	nama	String	Menyimpan informasi nama mata kuliah praktikum yang digunakan untuk keperluan registrasi dan operasional laboratorium.
	kalkulatorMap	Map<String, KalkulasiJalurUKT>	Menyimpan pemetaan jalur masuk mahasiswa ke objek kalkulator UKT yang sesuai, menggantikan struktur if-else pada kode awal (OCP).
	uktDasar	double	Menyimpan nilai dasar UKT per SKS yang digunakan sebagai basis perhitungan oleh seluruh implementasi KalkulasiJalurUKT.
	dataKRS	Map<String, List<OperasiMataKuliah>>	Menyimpan daftar mata kuliah yang diambil oleh setiap mahasiswa berdasarkan NIM secara dinamis selama program berjalan.

III. Constructor Dan Method

No	Nama Methode	Jenis Methode	Fungsi
1.	SistemKRSManager(DatabaseConnection db)	Constructor	Berperan sebagai wadah Constructor Injection untuk menyuntikkan objek database dari luar secara dinamis (DIP), menggantikan instansiasi kaku new MySQLDatabaseConnection() pada kode awal.
2.	kalkulasiUKT(KalkulasiJalurUKT jalurMasuk)	Functional	Mendelegasikan proses perhitungan UKT secara polimorfis tanpa struktur if-else berdasarkan objek strategi yang dikirim (OCP). Metode ini tidak perlu dimodifikasi saat jalur baru ditambahkan.
3.	simpanKRSKeDatabase(String mahasiswa, String matakuliah)	Procedural	Memerintah objek abstraksi db untuk membuka koneksi dan mengamankan riwayat pengambilan KRS ke media penyimpanan tanpa mengetahui detail teknis database yang digunakan.
4.	Hitung()	Functional	Kontrak metode polimorfisme pada interface KalkulasiJalurUKT yang wajib diimplementasikan oleh

			setiap variasi jalur masuk mahasiswa (Reguler, Karyawan, MBKM, dll).
5.	validasiPrasyaratMataKuliah(String mahasiswa, String matakuliah)	Functional	Melakukan pengecekan kelayakan akademik mahasiswa sebelum diizinkan mengambil suatu kelas. Diisolasi di kelas ValidatorAkademik sesuai prinsip SRP.
6.	cetakDraftKRS(String mahasiswa, String matakuliah)	Procedural	Menangani seluruh urusan visual presentasi dan pembuatan dokumen draf KRS. Diisolasi di kelas CetakKRSManger sesuai prinsip SRP agar tidak mencampuri logika penyimpanan data.
7.	registrasiKelas()	Procedural	Kontrak dasar pendaftaran kelas pada interface OperasiMataKuliah yang dijamin aman untuk dieksekusi oleh seluruh jenis mata kuliah tanpa memicu crash (LSP).
8.	alokasiAsistenLab()	Procedural	Kontrak spesifik pada interface OperasiPraktikum untuk menugaskan asisten laboratorium, hanya diimplementasikan oleh mata kuliah yang memiliki fasilitas praktikum (ISP).
9.	cekPeralatanPraktikum()	Procedural	Kontrak spesifik pada interface OperasiPraktikum untuk melakukan pengecekan inventaris alat lab sebelum praktikum dimulai (ISP).
10.	hubungkan()	Procedural	Kontrak abstraksi tingkat rendah pada interface DatabaseConnection untuk menginisialisasi jembatan komunikasi ke server database (DIP).
11.	simpanKRS(String mahasiswa, String matakuliah)	Functional	Kontrak abstraksi tingkat rendah pada interface DatabaseConnection untuk melakukan operasi insert data KRS mahasiswa ke media penyimpanan (DIP).
12.	daftarMahasiswa(String nim, String nama, String jalur)	Functional	Mendaftarkan mahasiswa baru ke dalam sistem dengan validasi NIM dan nama melalui ValidatorAkademik sebelum data disimpan.

IV. DOKUMENTASI DAN PEMBAHASAN KODE

Kode Awal Sebelum SOLID

```
2 class SistemKRSManager {
3     private MySQLDatabaseConnection db;
4     public SistemKRSManager() {
5     }
6
7     public boolean validasiPrasyaratMatakuliah(String mahasiswa, String matakuliah) {
8         System.out.println("Memvalidasi prasyarat akademik untuk " + mahasiswa);
9         return true;
10    }
11
12    public double hitungUangKuliahTunggal(String jalurMasuk) {
13        if (jalurMasuk.equalsIgnoreCase("Reguler")) {
14            return 5000000;
15        } else if (jalurMasuk.equalsIgnoreCase("Karyawan")) {
16            return 7500000;
17        } else if (jalurMasuk.equalsIgnoreCase("Internasional")) {
18            return 15000000;
19        } else if (jalurMasuk.equalsIgnoreCase("Bidikmisi")) {
20            return 0;
21        } else {
22            return 9000000;
23        }
24    }
25
26    public void cetakDraftKRS(String mahasiswa, String matakuliah) {
27        System.out.println("Proses cetak...");
28        boolean adaKesalahanFormatKop = true;
29        if (adaKesalahanFormatKop) {
30            System.out.println("Format Kop Surat Rusak! Proses dihentikan.");
31            return;
32        }
33        db.hubungkan();
34        db.simpanKRS(mahasiswa, matakuliah);
35    }
36 }
```

Penjelasan: Kelas SistemKRSManager di atas bertindak sebagai kelas terpusat karena memikul lebih dari satu tanggung jawab operasional. Kelas ini mencampuradukkan logika bisnis validasi akademik, kalkulasi keuangan (UKT), pembuatan dokumen presentasi (cetak draf PDF), hingga manajemen koneksi database tingkat rendah. Ketiadaan batasan tanggung jawab yang jelas memicu interdependensi tinggi. Hal ini terbukti pada metode cetakDraftKRS(), di mana kegagalan sepele pada format tampilan draf PDF (adaKesalahanFormatKop = true) secara langsung memotong alur program (return) dan menggagalkan eksekusi penyimpanan data ke database. Perubahan pada satu aspek presentasi merusak stabilitas aspek persistensi data.

Selain itu, metode hitungUangKuliahTunggal() melanggar OCP karena mengandalkan struktur seleksi kondisi if-else bertumpuk untuk menentukan nominal UKT berdasarkan parameter teks hardcoded. Arsitektur ini bersifat kaku (rigid) dan rapuh (fragile). Ketika pihak Rektorat menuntut penambahan skema perhitungan SKS baru untuk program "Merdeka Belajar Kampus Merdeka" (MBKM), pengembang terpaksa harus masuk dan memodifikasi langsung tubuh metode ini. Aktivitas membongkar kelas inti yang sudah stabil sangat berisiko merusak rumus kalkulasi jalur masuk lama akibat kesalahan manusiawi (human error).

```
1 public abstract class OperasiMatakuliah {
2     private String kode;
3     private String nama;
4     private int sks;
5
6     public OperasiMatakuliah(String kode, String nama, int sks) {
7         if (kode == null || kode.isBlank())
8             throw new IllegalArgumentException("Kode matakuliah tidak boleh kosong.");
9         if (sks <= 0)
10            throw new IllegalArgumentException("SKS harus lebih dari nol.");
11         this.kode = kode;
12         this.nama = nama;
13         this.sks = sks;
14     }
15
16     public String getKode() { return kode; }
17     public String getNama() { return nama; }
18     public int getSks() { return sks; }
19
20     public abstract String getJenis();
21
22     @Override
23     public String toString() {
24         return String.format("[%s] %s - %s (%d SKS)", kode, getJenis(), nama, sks);
25     }
26 }
```

Penjelasan: Antarmuka OperasiMataKuliah dirancang terlalu gemuk (Fat Interface), sehingga melanggar prinsip ISP karena memaksa kelas konsumen mengimplementasikan metode yang tidak mereka butuhkan. Akibat polusi antarmuka ini, kelas MataKuliahTeori dan Kuliah Lapangan (KKN) dipaksa mengimplementasikan metode alokasiAsistenLab() dan cekPeralatanPraktikum(). Karena karakteristik aslinya tidak mendukung operasional laboratorium, kelas turunan memanipulasi metode tersebut dengan melemparkan UnsupportedOperationException. Ketika sistem utama melakukan iterasi data secara polimorfis menggunakan tipe induk OperasiMataKuliah, pemanggilan metode lab pada objek MataKuliahTeori akan memicu runtime crash mendadak yang melumpuhkan sistem.

```
// Kode awal SistemKRSManager = sistem yang menggunakan banyak metode di satu kelas
class SistemKRSManager {
    private MySQLDatabaseConnection db;
    public SistemKRSManager() {
        // ...
    }
    public boolean validasiPrasyaratMataKuliah(String mahasiswa, String matakuliah) {
        System.out.println("Memvalidasi prasyarat akademik untuk " + mahasiswa);
        return true;
    }
    public double hitungUangKuliahTunggal(String jalurMasuk) {
        if (jalurMasuk.equalsIgnoreCase("Reguler")) {
            return 5000000;
        } else if (jalurMasuk.equalsIgnoreCase("Karyawan")) {
            return 7500000;
        } else if (jalurMasuk.equalsIgnoreCase("Internasional")) {
            return 15000000;
        } else if (jalurMasuk.equalsIgnoreCase("Bidikmisi")) {
            return 0;
        } else {
            return 9000000;
        }
    }
}
```

Penjelasan: Arsitektur di atas melanggar prinsip DIP karena modul tingkat tinggi (SistemKRSManager) bergantung secara langsung dan kaku (Tight Coupling) pada modul tingkat rendah konkret (MySQLDatabaseConnection). Hubungan dipererat secara paksa melalui instansiasi langsung menggunakan kata kunci new di dalam konstruktor. Kondisi ini membuat tim pengembang mengalami kelumpuhan teknis saat Rektorat meminta migrasi sistem ke infrastruktur Cloud NoSQL. Karena tidak adanya lapisan abstraksi, migrasi database tidak dapat dilakukan seperti sistem plug-and-play, melainkan harus merombak total seluruh deklarasi tipe data dan konstruktor di dalam komponen logika bisnis utama.

Penerapan Single Responsibility Principle (SRP)

Prinsip SRP menetapkan bahwa setiap kelas hanya boleh memikul satu tanggung jawab spesifik di dalam sistem. Berdasarkan hasil audit kode awal, kelas raksasa SistemKRSManager didekonstruksi dengan mengeluarkan fungsi validasi akademik dan fungsi manajemen cetak dokumen ke dalam dua kelas baru yang independen.

1. Kelas ValidatorAkademik

```
1 public class ValidatorAkademik {
2     private static final int MAKS_SKS = 24;
3     private static final int MIN_SKS = 1;
4
5     public void validasiSks(int totalSks) {
6         if (totalSks < MIN_SKS)
7             throw new IllegalArgumentException(
8                 "Total SKS tidak boleh kurang dari " + MIN_SKS + ".");
9         if (totalSks > MAKS_SKS)
10            throw new IllegalArgumentException(
11                "Total SKS melebihi batas maksimum " + MAKS_SKS + " SKS.");
12    }
13
14    public void validasiNim(String nim) {
15        if (nim == null || nim.isBlank())
16            throw new IllegalArgumentException("NIM tidak boleh kosong.");
17    }
18
19    public void validasiNama(String nama) {
20        if (nama == null || nama.isBlank())
21            throw new IllegalArgumentException("Nama tidak boleh kosong.");
22    }
23 }
```

Penjelasan: Logika pengecekan kelayakan akademik mahasiswa kini diisolasi sepenuhnya di dalam kelas ini. Jika di masa mendatang terdapat perubahan regulasi syarat pengambilan SKS, tim pengembang cukup memodifikasi kelas ini tanpa menyentuh modul keuangan maupun modul database.

2. Kelas CetakKRSManger

```
1  import java.util.List;
2
3  public class CetakKRSManger {
4
5      public void cetakKRS(String nim, String nama, String jalur,
6          List<OperasiMatakuliah> daftarMK,
7          int totalSKs, double ukt, String statusKRS) {
8          System.out.println("\n KARTU RENCANA STUDI");
9          System.out.printf("NIM      : %s\n", nim);
10         System.out.printf("Nama       : %s\n", nama);
11         System.out.printf("Jalur UKT : %s\n", jalur);
12         System.out.println("Mata Kuliah yang Diambil:");
13         for (OperasiMatakuliah mk : daftarMK) {
14             System.out.println(" " + mk);
15         }
16         System.out.printf("Total SKS : %d\n", totalSKs);
17         System.out.printf("UKT       : Rp%.0f\n", ukt);
18         System.out.printf("Status    : %s\n", statusKRS);
19         System.out.println();
20     }
21
22     public void cetakPesanBerhasil(String pesan) {
23         System.out.println("[OK] " + pesan);
24     }
25
26     public void cetakPeringatan(String pesan) {
27         System.out.println("Peringatan: " + pesan);
28     }
29 }
```

Penjelasan: Tanggung jawab memformat tampilan dan mencetak draf KRS dialihkan sepenuhnya ke kelas ini. Melalui pemisahan ini, kode manipulasi string tampilan tidak lagi bercampur dengan logika transaksi data.

3. Kelas SistemKRSManger

```
class SistemKRSManger {
    private MySQLDatabaseConnection db;
    public SistemKRSManger() {
    }
    public boolean validasiPrasyaratMatakuliah(String mahasiswa, String matakuliah) {
        System.out.println("Memvalidasi prasyarat akademik untuk " + mahasiswa);
        return true;
    }
    public double hitungUangKuliahTunggal(String jalurMasuk) {
        if (jalurMasuk.equalsIgnoreCase("Reguler")) {
            return 5000000;
        } else if (jalurMasuk.equalsIgnoreCase("Karyawan")) {
            return 7500000;
        } else if (jalurMasuk.equalsIgnoreCase("Internasional")) {
            return 15000000;
        } else if (jalurMasuk.equalsIgnoreCase("Bidikmisi")) {
            return 0;
        } else {
            return 9000000;
        }
    }
    public void cetakDraftKRS(String mahasiswa, String matakuliah) {
        System.out.println("Proses Cetak...");
        boolean adaKesalahanFormatKop = true;
        if (adaKesalahanFormatKop) {
            System.out.println("Format Kop Surat Rusak! Proses dihentikan.");
            return;
        }
        db.hubungkan();
        db.simpanKRS(mahasiswa, matakuliah);
    }
}
```

Penjelasan: Setelah fungsi validasi dan cetak dikeluarkan, kelas ini menjadi jauh lebih ramping. Metode cetakDraftKRS() pada kode awal kini digantikan oleh metode bersih simpanKRSSKeDatabase().

Penerapan Open/Closed Principle (OCP)

Prinsip OCP menegaskan bahwa sebuah komponen perangkat lunak harus terbuka untuk diperluas (open for extension), namun tertutup dari modifikasi langsung (closed for modification). Penumpukan struktur seleksi if-else pada metode lama `hitungUangKuliahTunggal()` telah dieliminasi total.

1. Interface

```
1 public interface KalkulasiJalurUKT {
2     double hitung(int totalSKS, double uktDasar);
3     String getNamaJalur();
4 }
```

Penjelasan: Setiap skema atau jalur masuk mahasiswa baru kini diwajibkan menjadi objek mandiri yang mengimplementasikan metode `hitung()` secara spesifik.

2. Enkapsulasi per Jalur

```
2 class UKTBidikmisi implements KalkulasiJalurUKT {
3     UKTBidikmisi() {
4     }
5
6     public double hitung() {
7         return (double)0.0F;
8     }
9 }
10
```

```
2 class UKTInternasional implements KalkulasiJalurUKT {
3     UKTInternasional() {
4     }
5
6     public double hitung() {
7         return (double)1.5E7F;
8     }
9 }
10
```

```
2 class UKTKaryawan implements KalkulasiJalurUKT {
3     UKTKaryawan() {
4     }
5
6     public double hitung() {
7         return (double)750000.0F;
8     }
9 }
10
```

```
2 class UKTMBKM implements KalkulasiJalurUKT {
3     UKTMBKM() {
4     }
5
6     public double hitung() {
7         return (double)250000.0F;
8     }
9 }
10
```

```
2 class UKTReguler implements KalkulasiJalurUKT {
3     UKTReguler() {
4     }
5
6     public double hitung() {
7         return (double)500000.0F;
8     }
9 }
10
```

Penjelasan: Seluruh isi percabangan logika dari kode awal diurai secara terpisah menjadi kelas konkret masing-masing agar rumus kalkulasi keuangan tiap jalur terisolasi dengan rapi.

Penerapan Liskov Substitution Principle (LSP) & Interface Segregation Principle (ISP)

Prinsip ISP menyatakan bahwa sebuah antarmuka tidak boleh memaksa kelas konsumen untuk mengimplementasikan metode yang tidak digunakannya, sedangkan LSP menjamin bahwa setiap subkelas harus dapat menggantikan peran tipe induknya secara transparan tanpa merusak kestabilan program.

1. ISP

```
1 public abstract class OperasiMataKuliah {
2     private String kode;
3     private String nama;
4     private int sks;
5
6     public OperasiMataKuliah(String kode, String nama, int sks) {
7         if (kode == null || kode.isBlank())
8             throw new IllegalArgumentException("Kode mata kuliah tidak boleh kosong.");
9         if (sks <= 0)
10            throw new IllegalArgumentException("SKS harus lebih dari nol.");
11         this.kode = kode;
12         this.nama = nama;
13         this.sks = sks;
14     }
15
16     public String getKode() { return kode; }
17     public String getNama() { return nama; }
18     public int getSks() { return sks; }
19
20     public abstract String getJenis();
21
22     @Override
23     public String toString() {
24         return String.format("[%s] %s - %s (%d SKS)", kode, getJenis(), nama, sks);
25     }
26 }
```

```
1 public interface OperasiPraktikum {
2     void alokasiAsistenLab();
3     void cekPeralatanPraktikum();
4 }
```

Penjelasan: Antarmuka induk OperasiMataKuliah dipangkas sehingga hanya menyisakan metode umum registrasiKelas(). Perilaku khusus terkait laboratorium diisolasi ke dalam antarmuka baru bernama OperasiPraktikum.

2. LSP

```
1 public class MataKuliahTeori extends OperasiMataKuliah {
2     public MataKuliahTeori(String kode, String nama, int sks) {
3         super(kode, nama, sks);
4     }
5
6     @Override
7     public String getJenis() { return "Teori"; }
8 }
```

MataKuliahTeori Bersih: Metode paksaan alokasiAsistenLab() dan cekPeralatanPraktikum() yang melanggar kontrak dihapus total. Kelas ini sekarang murni mengimplementasikan metode registrasiKelas() yang merupakan domain kapabilitas aslinya.

3. LSP & ISP

```
1 public class MataKuliahPraktikum extends OperasiMataKuliah implements OperasiPraktikum {
2     private String namaLab;
3     private String peralatan;
4
5     public MataKuliahPraktikum(String kode, String nama, int sks,
6     String namaLab, String peralatan) {
7         super(kode, nama, sks);
8         this.namaLab = namaLab;
9         this.peralatan = peralatan;
10    }
11
12    @Override
13    public void alokasiAsistenLab() {
14        System.out.println("Asisten lab dialokasikan untuk: " + namaLab);
15    }
16
17    @Override
18    public void cekPeralatanPraktikum() {
19        System.out.println("Peralatan dicek: " + peralatan);
20    }
21
22    @Override
23    public String getJenis() { return "Praktikum"; }
24 }
```

MataKuliahPraktikum dengan **Multiple Interface**: Kelas ini menggunakan kapabilitas multiple interface implementation Java dengan mengimplementasikan kedua antarmuka secara bersamaan tanpa melanggar prinsip desain apa pun.

Penerapan Dependency Inversion Principle (DIP)

Prinsip DIP menegaskan bahwa modul tingkat tinggi (logika bisnis utama) tidak boleh bergantung langsung pada modul tingkat rendah (detail teknis/infrastruktur), melainkan keduanya wajib bergantung pada sebuah lapisan abstraksi.

1. Interface Database

```
1 interface DatabaseConnection {
2     void hubungkan();
3     void simpanKRS(String mahasiswa, String matakuliah);
4 }
```

Penjelasan: Kontrak abstraksi baru ini menetapkan fungsi dasar sistem persistensi data tanpa terikat pada spesifikasi vendor database tertentu.

2. MySQLDatabaseConnection

```
1 public class MySQLDatabaseConnection implements DatabaseConnection {
2     @Override
3     public void hubungkan() {
4         System.out.println("Menghubungkan ke MySQL Database Server Kampus...");
5     }
6
7     @Override
8     public void simpanKRS(String mahasiswa, String matakuliah) {
9         System.out.println("DATA TERSIMPAN: KRS Mahasiswa " + mahasiswa
10         + " dengan kelas [" + matakuliah + "] sukses masuk tabel MySQL.");
11     }
12
13     @Override
```

Penjelasan: Kelas infrastruktur lama dikonfigurasi ulang untuk mengimplementasikan interface DatabaseConnection.

3. CloudNoSQLDatabaseConnection

```
2 public class CloudNoSQLDatabaseConnection implements DatabaseConnection {
3     @Override
4     public void hubungkan() {
5         System.out.println("Menghubungkan ke Cloud NoSQL Database...");
6     }
7
8     @Override
9     public void simpanKRS(String mahasiswa, String matakuliah) {
10        System.out.println("DATA TERSIMPAN: KRS Mahasiswa " + mahasiswa
11        + " sukses masuk Cluster NoSQL.");
12    }
13
14    @Override
```

Tuntutan Rektorat untuk migrasi ke Cloud NoSQL dipenuhi hanya dengan membuat kelas baru ini, tanpa mengubah satu baris pun kode SistemKRSManger.

4. Constructor Injection di SistemKRSManger

```
public SistemKRSManger(DatabaseConnection db, double uktDasar) {  
    this.db = db; // Injeksi dari luar – bukan new MySQLDatabaseConnection()  
}
```

Variabel referensi database diubah tipenya dari kelas konkret menjadi tipe abstraksi DatabaseConnection. Instansiasi kaku new MySQLDatabaseConnection() di dalam konstruktor dihapus total dan digantikan dengan mekanisme Constructor Injection.

Main.java (Demonstrasi Lengkap Seluruh Prinsip SOLID)

```
1 public class Main {  
2     public static void main(String[] args) {  
3  
4         // — Setup koneksi database (mudah diganti ke NoSQL) —  
5         DatabaseConnection db = new MySQLDatabaseConnection("jdbc:mysql://localhost:3306/siakad");  
6         SistemKRSManger sistem = new SistemKRSManger(db, 500.000);  
7         sistem.connect();  
8  
9         // — Daftarkan kalkulator UKT per jalur —  
10        sistem.daftarkanKalkulatorUKT(new KalkulasiJalurUKT() {  
11            public double hitung(int sks, double dasar) { return dasar * sks; }  
12            public String getNamaJalur() { return "REGULER"; }  
13        });  
14        sistem.daftarkanKalkulatorUKT(new KalkulasiJalurUKT() {  
15            public double hitung(int sks, double dasar) { return dasar * sks * 0.90; }  
16            public String getNamaJalur() { return "KARYAWAN"; }  
17        });  
18        sistem.daftarkanKalkulatorUKT(new KalkulasiJalurUKT() {  
19            public double hitung(int sks, double dasar) { return 0; }  
20            public String getNamaJalur() { return "BIDIKMISI"; }  
21        });  
22        sistem.daftarkanKalkulatorUKT(new KalkulasiJalurUKT() {  
23            public double hitung(int sks, double dasar) { return dasar * sks * 1.50; }  
24            public String getNamaJalur() { return "INTERNASIONAL"; }  
25        });  
26        // Jalur MBKM baru – cukup tambah di sini, tidak ada kode lain yang diubah  
27        sistem.daftarkanKalkulatorUKT(new KalkulasiJalurUKT() {  
28            public double hitung(int sks, double dasar) { return dasar * sks * 0.75; }  
29            public String getNamaJalur() { return "MBKM"; }  
30        });  
31  
32        // — Data mata kuliah —  
33        MataKuliahTeori mkTeori = new MataKuliahTeori("IF201", "Algoritma & Pemrograman", 3);  
34        MataKuliahPraktikum mkPrak = new MataKuliahPraktikum("IF202", "Praktikum Jaringan", 2, "Lab Jaringan", "Switch, Router, Kabel UTP");  
35        MataKuliahTeori mkWeb = new MataKuliahTeori("IF203", "Pemrograman Web", 3);  
36  
37        System.out.println("=== DATABASE: " + sistem.getDatabaseType() + " ===\n");  
38  
39        // — Daftarkan mahasiswa —  
40        sistem.daftarkanMahasiswa("22001", "Andi Saputra", "REGULER");  
41        sistem.daftarkanMahasiswa("22002", "Siti Rahma", "BIDIKMISI");  
42        sistem.daftarkanMahasiswa("22003", "Budi Santoso", "MBKM");  
43  
44        // — Tambah mata kuliah ke KRS —  
45        System.out.println();  
46        sistem.tambahMataKuliah("22001", mkTeori);  
47        sistem.tambahMataKuliah("22001", mkPrak);  
48        sistem.tambahMataKuliah("22001", mkWeb);  
49  
50        sistem.tambahMataKuliah("22002", mkTeori);  
51        sistem.tambahMataKuliah("22002", mkWeb);  
52    }  
53 }
```

Penjelasan: bagian main pada kode ini berfungsi sebagai simulasi untuk menguji hasil perbaikan (refactoring) arsitektur SIAKAD agar sesuai dengan prinsip SOLID. Alurnya dimulai dari inisialisasi koneksi awal ke database MySQL dan pendaftaran sistem manajer, yang kemudian dilanjutkan dengan mendaftarkan berbagai kalkulator rumus perhitungan UKT untuk 5 jalur masuk (termasuk jalur baru MBKM) menggunakan *anonymous class*. Setelah data mata kuliah teori dan praktikum dibuat, program mensimulasikan operasional pengisian KRS mahasiswa, pengujian validasi batasan sistem (seperti deteksi NIM duplikat atau over-SKS), hingga diakhiri dengan pembuktian migrasi database secara instan ke infrastruktur Cloud NoSQL tanpa merombak ulang logika bisnis utamanya.

V. KESIMPULAN

Penerapan lima prinsip desain SOLID pada sistem portal akademik SIAKAD terbukti mentransformasikan arsitektur aplikasi menjadi sangat tangkas dan modular. Restrukturisasi berbasis Single Responsibility Principle (SRP) sukses mengisolasi kegagalan operasional, sehingga kesalahan kecil pada aspek presentasi draf PDF tidak akan pernah lagi menginterupsi alur penyimpanan data ke dalam database. Selain itu, penghapusan tumpukan if-else menggunakan Open/Closed Principle (OCP) memberikan fleksibilitas penuh bagi sistem untuk mengakomodasi regulasi baru seperti skema perhitungan SKS MBKM melalui pembuatan ekstensi kelas baru, tanpa risiko merusak rumus perhitungan mahasiswa lama.

Di sisi lain, perbaikan pada hierarki kelas menggunakan Liskov Substitution Principle (LSP) dan Interface Segregation Principle (ISP) dengan memecah antarmuka besar menjadi kontrak yang spesifik berhasil mengeliminasi kemunculan runtime crash akibat `UnsupportedOperationException` secara permanen saat sistem melakukan iterasi data mata kuliah. Implementasi Dependency Inversion Principle (DIP), di mana ketergantungan kaku logika bisnis terhadap database MySQL telah diputus menggunakan lapisan abstraksi dan mekanisme dependency injection, membuat tuntutan migrasi infrastruktur menuju Cloud NoSQL dapat dieksekusi secara instan dan elegan tanpa perlu membongkar atau memodifikasi satu baris kode pun pada kelas pengelola utama.